

Lecture 5: Syntax Analysis Bottom-Up

中文学习笔记

基于课件 5.Syntax_Analysis_Bottom_Up.pdf 整理

2026 年 6 月 26 日

目录

阅读导引	4
1 课程总览：Bottom-up 到 LR Family [p1–3]	4
2 自底向上分析与移入-规约 [p4–9]	5
2.1 Bottom-up Parsing 的基本思想 [p4–5]	5
2.2 Shift-Reduce Parser 的四种动作 [p6–7]	5
2.3 例子：分析 <i>abbcd</i> [p8–9]	6
3 句柄、短语、直接短语与活前缀 [p10–27]	7
3.1 右句型、短语、直接短语、句柄 [p10–12]	7
3.2 句柄规约示例 [p13–24]	8
3.3 为什么句柄总在栈顶 [p25]	8
3.4 活前缀 [p26–27]	8
4 冲突与自底向上分析性质 [p28–31]	9
4.1 冲突来自哪里 [p28–30]	9
4.2 Bottom-up Parsing 的性质 [p31]	9
5 LR Parser 与 ACTION/GOTO 表 [p32–62]	9
5.1 LR(k) Parser 的含义 [p33–34]	9
5.2 LR Parser 的栈与右句型 [p35]	10
5.3 ACTION 表和 GOTO 表 [p37–41]	10

5.4	表驱动 LR 程序 [p42–44]	11
5.5	示例：用 LR 表分析 $1 + 1$ [p45–58]	11
5.6	解析表类型概览 [p59–62]	12
6	LR(0)：项目、自动机与解析表 [p63–95]	12
6.1	状态为什么需要 item [p63–66]	12
6.2	增广文法 [p67]	13
6.3	CLOSURE 与 GOTO [p68–74]	13
6.4	例子： $S \rightarrow BB$, $B \rightarrow aB \mid b$ [p69–78]	14
6.5	由 DFA 构造 LR(0) 表 [p77–80]	14
6.6	LR(0) 冲突与局限 [p81–95]	15
7	SLR(1)：用 FOLLOW 限制规约 [p96–106]	15
7.1	SLR(1) 的基本思想 [p98–99]	15
7.2	SLR(1) 文法判定条件 [p100]	16
7.3	SLR(1) 的优势与局限 [p101–106]	16
8	LR(1)：把 Lookahead 放进 Item [p107–122]	17
8.1	LR(1) item [p107–108]	17
8.2	LR(1) 的 CLOSURE 和 GOTO [p109–110]	17
8.3	LR(1) 示例 [p109–121]	18
8.4	LR(1) 解析表与文法条件 [p120–122]	18
9	LALR(1)：合并同 Core 状态的折中方案 [p123–139]	19
9.1	为什么需要 LALR(1) [p124]	19
9.2	状态合并示例 [p125–127]	19
9.3	合并效果：冲突与错误延迟 [p128–133]	20
9.4	LALR 表构造与文法关系 [p134–137]	20
9.5	LL vs LR [p138–139]	21
10	Syntax Analysis Revisit：语法分析总复习 [p140–189]	21
10.1	语法分析的位置与文法定义 [p141–145]	21
10.2	句型、句子、语言、二义性 [p144–145]	22

目录	3
10.3 乔姆斯基文法体系与 AST [p146–148]	22
10.4 Top-down 复习: RDP、FIRST、FOLLOW 与 LL(1) [p149–160]	22
10.5 Bottom-up 复习 [p161–189]	23
11 练习题与解答 [p190–206]	24
复习建议	27

阅读导引

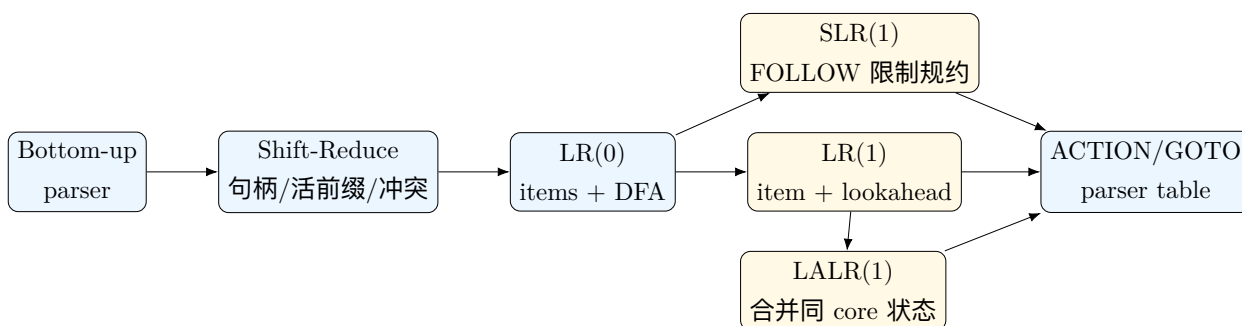
本讲围绕语法分析的自底向上路线展开，核心是 LR 系列分析器。前半部分讲如何从移入-规约分析走到 LR(0) 自动机和解析表；中间依次引入 SLR(1)、LR(1)、LALR(1) 来解决 LR(0) 过弱和 LR(1) 表过大的问题；后半部分复习前几讲语法分析内容，并给出练习题讲解。

一条主线

自底向上分析的根本问题是：栈顶什么时候已经形成句柄，应该用哪条产生式规约。LR 系列的解决思路是把“当前已经识别到哪里”编码成状态，再用 ACTION/GOTO 表驱动移入和规约。

1 课程总览：Bottom-up 到 LR Family [p1-3]

课件开头给出全局路线图：



LR 系列的几个变体可以先这样理解：

类型	使用的信息	规约策略	特点
LR(0)	只有 LR(0) item 和栈状态	完成项出现就规约	最简单、最弱，容易冲突
SLR(1)	LR(0) 状态 + FOLLOW 集	下一个输入符号在 FOLLOW(A) 中才用 $A \rightarrow \alpha$ 规约	状态数与 LR(0) 相同，能力更强
LR(1)	LR(0) item + 精确 lookahead	下一个输入符号等于 item 的 lookahead 才规约	能力强，但状态数和表可能很大
LALR(1)	LR(1) lookahead + 合并同 core 状态	保留 lookahead，但减少状态数	实用折中，YACC/Bison 常用路线

2 自底向上分析与移入-规约 [p4-9]

2.1 Bottom-up Parsing 的基本思想 [p4-5]

定义：自底向上分析

自底向上分析 (bottom-up parsing) 从输入串出发，逐步把某些子串替换为产生式左部的非终结符，直到整个输入被规约为开始符号。它等价于“最右推导的逆过程”，语法树是从叶子结点向根结点构造出来的。

通俗地说，Top-down 是“从开始符号猜出输入串”，Bottom-up 是“从输入串往回合成开始符号”。例如表达式文法：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid id$$

对输入 $id * id$ ，一种最右推导为

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id.$$

自底向上分析反过来做：

$$id * id \Rightarrow F * id \Rightarrow T * id \Rightarrow T * F \Rightarrow T \Rightarrow E.$$

自底向上分析相对自顶向下分析更强：

- 不要求对文法做左因子提取 (left factoring)。
- 可以处理左递归 (left recursion)，而递归下降分析直接遇到左递归会无限递归。
- 代价是手写分析器很困难，通常由工具根据文法自动生成。

2.2 Shift-Reduce Parser 的四种动作 [p6-7]

定义：移入-规约分析

Shift-reduce parsing 是自底向上分析的一种形式：栈保存已经读入的文法符号，输入缓冲区保存尚未扫描的输入串。分析器从左到右扫描输入，反复执行移入和规约。

动作	含义
----	----

Shift 移入	把下一个输入符号压入栈顶。通常表示句柄还没有完整出现在栈顶，需要继续读输入。
Reduce 规约	栈顶某段符号串匹配某条产生式右部 β ，就把它替换为产生式左部 A 。也就是执行 $\beta \Rightarrow A$ 的逆向合成。
Accept 接受	输入已经成功规约为开始符号，分析完成。
Error 报错	当前状态和输入符号无合法动作，说明语法错误，需要报错或恢复。

2.3 例子：分析 $abbcd e$ [p8-9]

给定文法：

$$S \rightarrow aAcBe, \quad A \rightarrow b, \quad A \rightarrow Ab, \quad B \rightarrow d.$$

输入为 $abbcd e$ 。课件展示的分析过程如下：

栈	输入	动作
\$	$abbcd e \$$	Shift
$\$a$	$bcd e \$$	Shift
$\$ab$	$cd e \$$	Reduce $A \rightarrow b$
$\$aA$	$cd e \$$	Shift
$\$aAb$	$d e \$$	Reduce $A \rightarrow Ab$
$\$aA$	$d e \$$	Shift
$\$aAc$	$e \$$	Shift
$\$aAcd$	$e \$$	Reduce $B \rightarrow d$
$\$aAcB$	$e \$$	Shift
$\$aAcBe$	$\$$	Reduce $S \rightarrow aAcBe$
$\$S$	$\$$	Accept

这个例子引出自底向上分析的两个核心决策：

- 什么时候应该继续 Shift，什么时候应该 Reduce？
- Reduce 时到底应该用哪条产生式？

例如栈顶出现 b 时，既可能用 $A \rightarrow b$ 规约，也可能继续读入，等待形成 Ab 再用 $A \rightarrow Ab$ 规约。解决这个问题需要引入“句柄”。

3 句柄、短语、直接短语与活前缀 [p10-27]

3.1 右句型、短语、直接短语、句柄 [p10-12]

右句型

右句型 (right-sentential form) 是在最右推导过程中出现的句型。自底向上分析做的是最右推导的逆过程，因此它关心当前输入对应的右句型。

短语、直接短语、句柄

设 $\alpha_1 A \alpha_2$ 是文法 $G(S)$ 的一个右句型，且

$$S \Rightarrow^* \alpha_1 A \alpha_2 \Rightarrow \alpha_1 \beta \alpha_2.$$

则 β 是该句型的一个短语 (phrase)。若这一步 $A \Rightarrow \beta$ 对应语法树中只有父子两代的子树，则 β 是直接短语 (simple phrase)。句柄 (handle) 是右句型中最左的直接短语。

通俗解释：

- 短语：语法树中某个子树的所有叶子拼成的串。
- 直接短语：该子树只有“父结点 - 子结点”两层，说明它刚好对应一条产生式右部。
- 句柄：当前最应该先规约的那一个直接短语。自底向上分析每一步都应该规约句柄。

对文法

$$E \rightarrow T \mid E + T, \quad T \rightarrow F \mid T * F, \quad F \rightarrow (E) \mid a \mid b \mid c$$

和句型 $a * b + c$ ，课件给出的结果为：

类别	符号串
短语	$a * b + c, a * b, a, b, c$
直接短语	a, b, c
句柄	a

注意 $b + c$ 、 $*b$ 、 $+$ 不是短语，因为它们不能对应语法树中某个完整子树的叶子序列。

3.2 句柄规约示例 [p13-24]

课件用句型 $T + T * F + id$ 展示句柄如何逐步被规约。对应表达式文法：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid id.$$

该句型中的短语包括：

$$T + T * F + id, \quad T + T * F, \quad T, \quad T * F, \quad id.$$

直接短语包括左侧的 T 、 $T * F$ 中的可规约部分，以及 id 等；每一步真正规约的是最左直接短语，即句柄。

核心规约序列可以概括为：

$$T + T * F + id \Rightarrow E + T * F + id \Rightarrow E + T + id \Rightarrow E + id \Rightarrow E + F \Rightarrow E + T \Rightarrow E.$$

在 $abbcd$ 的例子中，栈为 $\$aAb$ 、输入为 $cde\$$ 时，句柄是 Ab ，因此应选择 $A \rightarrow Ab$ 规约，而不是错误地只把 b 按 $A \rightarrow b$ 规约。

3.3 为什么句柄总在栈顶 [p25]

句柄可能一开始还在输入缓冲区中，没有完全进入栈；但分析器会继续 Shift，直到句柄完整出现在栈顶。一旦句柄在栈顶，分析器会立即 Reduce。因此句柄不会“长期卡在栈中间”。

这给出通用移入-规约策略：

- 若栈顶没有句柄，Shift。
- 若栈顶已经形成句柄，Reduce 成相应非终结符。
- 可通过表自动化实现上述决策。

3.4 活前缀 [p26-27]

活前缀

活前缀 (viable prefix) 是某个右句型的前缀，并且该前缀没有越过该右句型最右句柄的右端。

直观上，活前缀是“仍然有希望成为正确规约路径的栈内容”。LR 分析过程中，栈中符号串始终是某个右句型的活前缀，因此分析器不会偏离正确方向。

课件例子：

$$S \Rightarrow aEb \Rightarrow aaEb.$$

若句柄为 aE ，则活前缀包括 a, aa, aaE ，但不包括 $aaEb$ ，因为 $aaEb$ 已经越过句柄右端。

另一个例子中，右句型 $aAcde$ 可被分在栈和输入之间；栈内容 aA 、 aAc 、 $aAcd$ 都是活前缀。

4 冲突与自底向上分析性质 [p28-31]

4.1 冲突来自哪里 [p28-30]

考虑二义表达式文法：

$$E \rightarrow E * E \mid E + E \mid (E) \mid id.$$

对输入 $id * id + id$ ，当栈中已经有 $E * E$ 、下一个输入符号是 $+$ 时，有两种选择：

- 先 Reduce $E \rightarrow E * E$ ，得到 $(id * id) + id$ ，体现 $*$ 优先于 $+$ 。
- 先 Shift $+$ ，之后再规约，得到 $id * (id + id)$ 。

文法没有说明优先级和结合性，所以两个选择都看似合理。这就是冲突。

两类冲突

Shift-reduce conflict：同一状态下既可以移入，也可以规约。Reduce-reduce conflict：同一状态下可用两条或多条产生式规约。

大多数冲突来自二义文法，但也存在非二义文法无法被某类弱分析器处理的情况。处理冲突的常见方式包括：

- 改写文法，把运算符优先级和结合性编码进去。
- 对特殊结构手动消除二义性，例如 dangling-else。
- 使用更强的分析器类别，例如从 LR(0) 提升到 SLR(1)、LR(1) 或 LALR(1)。

4.2 Bottom-up Parsing 的性质 [p31]

课件总结了三点：

- 句柄总会出现于栈顶，这说明用栈实现移入-规约是合理的。
- 栈顶没有句柄就移入，栈顶有句柄就规约。
- 自动化的关键是构造表，让分析器知道每个状态和输入下应采取什么动作。

5 LR Parser 与 ACTION/GOTO 表 [p32-62]

5.1 LR(k) Parser 的含义 [p33-34]

课件列出多种自底向上分析器：simple precedence parser、operator precedence parser、recursive ascent parser、LR family parser 等。本讲主要关注 LR 系列。

LR(k)

LR(k) 是 LR 系列分析器。L 表示从左到右扫描输入 (left-to-right scanning)，R 表示反向构造最右推导 (rightmost derivation in reverse)，k 表示做语法决策时向前看的输入符号数量。

与 LL(k) 相比：

- 二者都可达到输入长度线性时间和线性空间。
- 二者都能由工具根据文法自动生成，例如 YACC、Bison。
- LR 比 LL 更强，不需要消除左递归，也不需要左因子提取。
- LR 冲突调试更复杂，因为冲突发生在自动机状态和表项中，不像 LL 那样直接对应某个递归函数选择。

5.2 LR Parser 的栈与右句型 [p35]

LR parser 的栈保存状态序列：

$$S_0 S_1 \cdots S_m,$$

其中 S_m 是栈顶状态。每个文法符号 X_i 会关联一个状态 S_i 。若输入串属于该语言，则“栈中符号 + 未读输入”构成某个右句型：

$$X_1 X_2 \cdots X_m a_i a_{i+1} \cdots a_n.$$

分析器根据栈顶状态 S_m 和当前输入符号 a_i 查表：

$$\text{ACTION}[S_m, a_i].$$

5.3 ACTION 表和 GOTO 表 [p37-41]

LR 解析表由两个部分组成：

表	功能
ACTION 表	按“状态 + 终结符”索引，决定移入、规约、接受或报错。
GOTO 表	按“状态 + 非终结符”索引，在规约后决定新的栈顶状态。

ACTION 的四种结果如下：

表项	执行方式
s_x	Shift 当前输入符号，并把状态 x 压栈。

r_y	Reduce。若第 y 条产生式为 $A \rightarrow \beta$ ，弹出 $ \beta $ 个符号及其状态，压入 A ，再查GOTO压入新状态。
acc	Accept，语法分析成功。
空表项	Error，输入不符合文法或需要错误恢复。

规约后的关键动作是：

$$S_j = \text{GOTO}[S_i, A],$$

其中 S_i 是弹出句柄后新的栈顶状态， A 是规约得到的非终结符。

5.4 表驱动 LR 程序 [p42-44]

LR 程序的伪代码可以概括为：

1. 初始时把 S_0 压栈，输入缓冲区为 $\omega\$$ 。
2. 令 s 为栈顶状态， a 为当前输入符号。
3. 若 $\text{ACTION}[s, a] = s_t$ ，移入 a 并压入状态 t 。
4. 若 $\text{ACTION}[s, a] = r_k$ ，设第 k 条产生式为 $A \rightarrow \beta$ ，弹出 $|\beta|$ 个符号和状态，压入 A ，再压入 $\text{GOTO}[s', A]$ ，其中 s' 是弹出后的栈顶状态。
5. 若 $\text{ACTION}[s, a] = acc$ ，接受。
6. 否则报错。

5.5 示例：用 LR 表分析 $1 + 1$ [p45-58]

文法为：

$$(1) E \rightarrow E * B, \quad (2) E \rightarrow E + B, \quad (3) E \rightarrow B, \quad (4) B \rightarrow 0, \quad (5) B \rightarrow 1.$$

课件给出解析表，并用它分析输入 $1 + 1$ 。关键步骤如下：

状态栈	符号栈	输入	动作/跳转
0	\$	1 + 1\$	s_2
0 2	\$1	+1\$	$r_5: B \rightarrow 1, \text{GOTO}(0, B) = 4$
0 4	\$B	+1\$	$r_3: E \rightarrow B, \text{GOTO}(0, E) = 3$
0 3	\$E	+1\$	s_6
0 3 6	\$E+	1\$	s_2
0 3 6 2	\$E+1	\$	$r_5: B \rightarrow 1, \text{GOTO}(6, B) = 8$
0 3 6 8	\$E+B	\$	$r_2: E \rightarrow E+B, \text{GOTO}(0, E) = 3$
0 3	\$E	\$	acc

这个例子说明：LR 分析器本身只负责查表执行动作；真正困难的是构造可靠的解析表，让每个状态和输入符号都能得到正确的 ACTION/GOTO 动作。

5.6 解析表类型概览 [p59-62]

课件将解析表构造分为几类：

- LR(0)：只看栈状态，不看任何输入展望符号。简单但太弱。
- SLR(1)：在 LR(0) 状态基础上，用 FIRST/FOLLOW 信息给规约加限制。
- LALR(1)：常用变体，用更精细的 lookahead 分析，但仍使用接近 LR(0) 的状态规模。
- LR(1)：精确的一符号 lookahead，算法和表更复杂。
- LR(k)：使用 k 个展望符号，表更大。

6 LR(0): 项目、自动机与解析表 [p63-95]

6.1 状态为什么需要 item [p63-66]

问题：当栈顶是 T 、下一个输入符号是 $*$ 时，分析器如何知道不能立刻把 T 规约成 E ，而应该先移入 $*$ ？

LR 的答案是维护状态。状态表示“分析已经走到哪里”，由一组 item 构成。

LR(0) item

LR(0) item 是在产生式右部某个位置加入点号的产生式。对 $A \rightarrow XYZ$ ，有四个 item：

$$A \rightarrow \cdot XYZ, \quad A \rightarrow X \cdot YZ, \quad A \rightarrow XY \cdot Z, \quad A \rightarrow XYZ \cdot.$$

若产生式为 $A \rightarrow \varepsilon$ ，只有一个 item： $A \rightarrow \cdot$ 。

点号的意义：

- $A \rightarrow \cdot XYZ$ ：希望接下来看到能由 XYZ 推导出的串。
- $A \rightarrow X \cdot YZ$ ：已经识别了 X ，接下来期望 YZ 。
- $A \rightarrow XYZ \cdot$ ：已经识别了完整右部 XYZ ，可能可以规约为 A 。

一个状态通常不是单个 item，而是一组 item，称为配置集 (configuration set)。例如如果有 item $A \rightarrow X \cdot YZ$ ，且 $Y \rightarrow u \mid w$ ，则当前状态也应该包含 $Y \rightarrow \cdot u$ 、 $Y \rightarrow \cdot w$ ，因为接下来可能要识别 Y 。

6.2 增广文法 [p67]

增广文法

为了让自动机有明确的接受状态，给原文法 G 增加新的开始符号 S' 和产生式 $S' \rightarrow S$ ，得到增广文法 G' 。当状态中出现 $S' \rightarrow S \cdot$ 且输入为\$ 时，分析接受。

例如原文法：

$$E \rightarrow E * B \mid E + B \mid B, \quad B \rightarrow 0 \mid 1$$

增广后为：

$$S \rightarrow E, \quad E \rightarrow E * B \mid E + B \mid B, \quad B \rightarrow 0 \mid 1.$$

6.3 CLOSURE 与 GOTO [p68–74]

CLOSURE

若 I 是 item 集， $\text{CLOSURE}(I)$ 按以下规则构造：

1. 初始把 I 中所有 item 加入闭包。
2. 若闭包中有 $A \rightarrow \alpha \cdot B\beta$ ，且文法中有 $B \rightarrow \gamma$ ，则加入 $B \rightarrow \cdot\gamma$ 。
3. 重复上述过程，直到没有新 item 可加入。

通俗解释：点号后面如果是非终结符 B ，说明下一步可能要识别一个 B 。既然要识别 B ，就必须把 B 的所有候选产生式都放进当前状态。

GOTO

$\text{GOTO}(I, X)$ 表示从 item 集 I 通过文法符号 X 能到达的新 item 集。具体做法是：找到所有 $A \rightarrow \alpha \cdot X\beta$ ，把点号移过 X 得到 $A \rightarrow \alpha X \cdot \beta$ ，再对这些 item 取闭包。

构造 LR(0) 状态集的算法：

1. 构造增广文法 G' 。
2. 初始状态为 $\text{CLOSURE}(\{S' \rightarrow \cdot S\})$ 。

3. 对每个已有状态 I 和每个文法符号 X , 计算GOTO(I, X)。
4. 若结果非空且是新状态, 加入状态集。
5. 重复直到不再产生新状态。

所有状态构成规范 LR(0) 项集族 (canonical LR(0) collection), 它本质上就是识别活前缀的 DFA。

6.4 例子: $S \rightarrow BB, B \rightarrow aB \mid b$ [p69-78]

增广文法:

$$(0) S' \rightarrow S, \quad (1) S \rightarrow BB, \quad (2) B \rightarrow aB, \quad (3) B \rightarrow b.$$

初始状态:

$$I_0 = \{S' \rightarrow \cdot S, S \rightarrow \cdot BB, B \rightarrow \cdot aB, B \rightarrow \cdot b\}.$$

若从 I_0 经过 B :

$$\text{GOTO}(I_0, B) = \text{CLOSURE}(\{S \rightarrow B \cdot B\}) = \{S \rightarrow B \cdot B, B \rightarrow \cdot aB, B \rightarrow \cdot b\}.$$

若从 I_0 经过 a :

$$\text{GOTO}(I_0, a) = \text{CLOSURE}(\{B \rightarrow a \cdot B\}) = \{B \rightarrow a \cdot B, B \rightarrow \cdot aB, B \rightarrow \cdot b\}.$$

若从 I_0 经过 b :

$$\text{GOTO}(I_0, b) = \{B \rightarrow b \cdot\}.$$

课件给出的 LR(0) 表可概括为:

State	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

6.5 由 DFA 构造 LR(0) 表 [p77-80]

构造规则:

- 若状态 S_i 中有 $A \rightarrow \alpha \cdot a\beta$, 其中 a 是终结符, 并且 $\text{GOTO}(S_i, a) = S_j$, 则 $\text{ACTION}[i, a] = s_j$ 。
- 若状态 S_i 中有完成项 $A \rightarrow \alpha \cdot$, 且该产生式是第 j 条, 则对所有终结符 a 填 $\text{ACTION}[i, a] = r_j$ 。
- 若状态中有 $S' \rightarrow S \cdot$, 则 $\text{ACTION}[i, \$] = acc$ 。

- 若 $\text{GOTO}(S_i, A) = S_j$, 其中 A 是非终结符, 则 $\text{GOTO}[i, A] = j$ 。

LR(0) 的“0”表示分析器不能看未来输入, 只能根据当前状态决定动作。因此状态必须非常干净: 要么只移入, 要么只规约, 不能混杂。

6.6 LR(0) 冲突与局限 [p81–95]

LR(0) 冲突条件:

- Reduce-reduce conflict: 某状态同时有两个完成项, 例如 $X \rightarrow \beta \cdot$ 和 $Y \rightarrow \omega \cdot$ 。
- Shift-reduce conflict: 某状态同时有完成项和可移入项, 例如 $X \rightarrow \beta \cdot$ 和 $Y \rightarrow \omega \cdot t$ 。

课件示例中, 文法:

$$E' \rightarrow E, \quad E \rightarrow E + T \mid T \mid V = E, \quad T \rightarrow (E) \mid i \mid i[E], \quad V \rightarrow i$$

在读入 i 后可能出现:

$$T \rightarrow i \cdot, \quad T \rightarrow i \cdot [E], \quad V \rightarrow i \cdot.$$

这里既有完成项, 又有点后为 $[$ 的可移入项, 并且有多个完成项, LR(0) 无法决定。

LR(0) 的根本局限

LR(0) 看到完成项就倾向于规约, 但当前输入符号可能说明“不该现在规约”。因此需要引入 lookahead。SLR(1) 用 FOLLOW 集做粗粒度限制; LR(1) 把精确 lookahead 放进 item。

7 SLR(1): 用 FOLLOW 限制规约 [p96–106]

7.1 SLR(1) 的基本思想 [p98–99]

SLR(1)

SLR(1) 即 Simple LR with 1-token lookahead。它使用与 LR(0) 相同的配置集、表结构和分析算法, 但在填 reduce 动作时加入 FOLLOW 集限制: 只有当当前输入符号属于 $\text{FOLLOW}(A)$ 时, 才允许用 $A \rightarrow \alpha$ 规约。

如果状态中有完成项 $A \rightarrow \alpha \cdot$, LR(0) 会把规约动作填到所有终结符列; SLR(1) 只把规约动作填到 $\text{FOLLOW}(A)$ 对应的列。这样很多无意义规约会被删除。

课件示例中:

$$\text{FOLLOW}(T) = \{+,),], \$\}, \quad \text{FOLLOW}(V) = \{=\}.$$

当读入 id 后:

- 若下一个输入符号在 $\text{FOLLOW}(T)$ 中, 规约为 T 。
- 若下一个输入符号在 $\text{FOLLOW}(V)$ 中, 规约为 V 。
- 若下一个输入符号是 $[$, 则移入, 继续识别 $id[E]$ 。

7.2 SLR(1) 文法判定条件 [p100]

一个文法是 SLR(1), 要求每个配置集满足:

1. 若集合中有 $A \rightarrow u \cdot x v$, 其中 x 是终结符, 则集合中不能存在完成项 $B \rightarrow w \cdot$ 且 $x \in \text{FOLLOW}(B)$ 。这对应无 shift-reduce conflict。
2. 若集合中有两个完成项 $A \rightarrow u \cdot$ 和 $B \rightarrow v \cdot$, 则 $\text{FOLLOW}(A) \cap \text{FOLLOW}(B) = \emptyset$ 。这对应无 reduce-reduce conflict。

通俗地说, 如果一个状态中有多个可能动作, 只要一个输入符号能唯一告诉我们该选哪一个, SLR(1) 就能处理。

7.3 SLR(1) 的优势与局限 [p101–106]

优势:

- 只加一个 lookahead 和 FOLLOW 集, 就大幅扩展了 LR(0) 能处理的文法范围。
- 仍然使用 LR(0) 项集, 所以状态数量不增加。
- 同一状态可以同时有移入项和规约项, 也可以有多个完成项, 只要 FOLLOW 能把动作区分开。

局限:

- FOLLOW 是全局信息, 只说明“某个非终结符在任何合法句子中可能跟随什么符号”。
- 当前状态只描述栈顶这段串, 不能充分记录栈下方的上下文。
- 因此有时 $\text{FOLLOW}(A)$ 包含某符号, 但在当前这个具体上下文中并不应该用 A 规约。

典型限制例子:

$$S \rightarrow L = R \mid R, \quad L \rightarrow *R \mid id, \quad R \rightarrow L.$$

输入 $id = id$ 时, 读入第一个 id 后可规约为 L 。在某状态中同时存在:

$$S \rightarrow L \cdot = R, \quad R \rightarrow L \cdot .$$

看到下一个输入符号 $=$ 时, 正确动作应是 shift, 因为要继续识别赋值式 $L = R$ 。但由于 $= \in \text{FOLLOW}(R)$, SLR(1) 也会允许 $R \rightarrow L$ 规约, 于是出现 shift-reduce conflict。

这说明：只靠 FOLLOW 不够，需要更精确地记录“当前完成项是在什么上下文里出现的”，这就是 LR(1) 的动机。

8 LR(1): 把 Lookahead 放进 Item [p107-122]

8.1 LR(1) item [p107-108]

LR(1) item

LR(1) item 的一般形式为：

$$[A \rightarrow X_1 \cdots X_i \cdot X_{i+1} \cdots X_j, a].$$

其中点号左边表示已经识别的部分，点号右边表示期待识别的部分， a 是 lookahead。若 item 已完成：

$$[A \rightarrow X_1 \cdots X_j \cdot, a],$$

则只有当下一个输入符号正好是 a 时，才允许规约。

对 reduce 条件的精度可以这样比较：

分析器	规约条件	精度
LR(0)	状态中有完成项就规约	最粗
SLR(1)	下一输入符号在 FOLLOW(A) 中才用 $A \rightarrow \alpha$ 规约	中等
LR(1)	下一输入符号等于 item 的 lookahead 才规约	最精确

LR(1) 的代价是状态变多：很多 LR(0) item 的 core 相同，但 lookahead 不同，因此会被区分为不同 LR(1) item，进而产生更多状态。

8.2 LR(1) 的 CLOSURE 和 GOTO [p109-110]

LR(1) 状态构造与 LR(0) 类似，但闭包和跳转要传播 lookahead。

LR(1) CLOSURE

若 I 中有 item $[A \rightarrow u \cdot Bv, a]$ ，且文法有 $B \rightarrow w$ ，则对每个

$$b \in \text{FIRST}(va)$$

加入 item $[B \rightarrow \cdot w, b]$ 。

为什么是 $\text{FIRST}(va)$? 因为当前正在期望一个 B , 而 B 后面紧跟 v , 整个 A 规约后又可能遇到 lookahead a 。因此 B 规约完成后可能看到的符号就是 va 能推出串的首符号。

LR(1) GOTO

若 I 中有 $[A \rightarrow u \cdot Xv, a]$, 则

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{[A \rightarrow uX \cdot v, a]\}).$$

和 LR(0) 一样移动点号, 但必须携带并传播 lookahead。

构造步骤:

1. 从 $\text{CLOSURE}([S' \rightarrow \cdot S, \$])$ 开始。
2. 对每个状态和文法符号计算GOTO。
3. 重复直到没有新状态。

8.3 LR(1) 示例 [p109–121]

文法:

$$(0) S' \rightarrow S, \quad (1) S \rightarrow XX, \quad (2) X \rightarrow aX, \quad (3) X \rightarrow b.$$

初始状态的一部分:

$$\begin{aligned} I_0 = \{ & S' \rightarrow \cdot S, \$; \\ & S \rightarrow \cdot XX, \$; \\ & X \rightarrow \cdot aX, a/b; \\ & X \rightarrow \cdot b, a/b \}. \end{aligned}$$

这里 X 的 lookahead 是 a/b , 因为在 $S \rightarrow \cdot XX$ 中, 第一个 X 后面还有一个 X , 而 $\text{FIRST}(X\$) = \{a, b\}$ 。

当进入 $S \rightarrow X \cdot X, \$$ 后, 第二个 X 后面直接是 $\$$, 所以新加入的 X 相关 item 的 lookahead 是 $\$$:

$$X \rightarrow \cdot aX, \$, \quad X \rightarrow \cdot b, \$.$$

这正是 LR(1) 比 SLR(1) 更精确的地方: 同样是 $X \rightarrow b$, 在不同状态中可能只在 a/b 下规约, 或只在 $\$$ 下规约。

8.4 LR(1) 解析表与文法条件 [p120–122]

LR(1) 表构造:

- 若 $[A \rightarrow \alpha \cdot a\beta, b] \in S_i$ 且 $\text{GOTO}(S_i, a) = S_j$, 则 $\text{ACTION}[i, a] = s_j$ 。Shift 不关心 lookahead。

- 若 $[A \rightarrow \alpha \cdot, a] \in S_i$, 则只在 $\text{ACTION}[i, a]$ 填入 reduce。
- GOTO 表仍由非终结符转移决定。

一个文法是 LR(1), 要求每个 LR(1) 配置集满足:

1. 对任意移入项 $[A \rightarrow u \cdot xv, a]$, 不存在完成项 $[B \rightarrow v' \cdot, x]$ 。这表示无 shift-reduce conflict。
2. 所有完成项的 lookahead 两两不冲突, 即不能同时有 $[A \rightarrow u \cdot, a]$ 和 $[B \rightarrow v \cdot, a]$ 。这表示无 reduce-reduce conflict。

每个 SLR(1) 文法都是 LR(1), 但 LR(1) parser 可能有更多状态, 因为 LR(1) 会按不同 lookahead 拆分状态。

9 LALR(1): 合并同 Core 状态的折中方案 [p123–139]

9.1 为什么需要 LALR(1) [p124]

LR(1) 的缺点是状态数可能远多于 SLR(1) 或 LR(0)。一个 LR(0) item 可能因为不同 lookahead 被拆成多个 LR(1) item, 理论上可能带来指数级状态增长。

LALR(1)

LALR(1) 即 lookahead LR。它先获得 LR(1) 的精确 lookahead 信息, 再合并“core 相同”的状态, 以减少状态数。它试图在 LR(1) 的表达能力和 SLR(1) 的表规模之间折中。

Core

LR(1) item 去掉 lookahead 后剩下的 LR(0) item 称为 core。若两个状态中的 item 数量相同, 且所有 item 的 core 相同, 只是 lookahead 不同, 则这些状态可以尝试合并。

9.2 状态合并示例 [p125–127]

课件中, LR(1) 状态:

$$I_3 : X \rightarrow a \cdot X, a/b; X \rightarrow \cdot aX, a/b; X \rightarrow \cdot b, a/b,$$

$$I_6 : X \rightarrow a \cdot X, \$; X \rightarrow \cdot aX, \$; X \rightarrow \cdot b, \$.$$

二者 core 相同, 只是 lookahead 不同, 因此合并为:

$$I_{36} : X \rightarrow a \cdot X, a/b/\$; X \rightarrow \cdot aX, a/b/\$; X \rightarrow \cdot b, a/b/\$.$$

类似地, I_4 与 I_7 合并为 I_{47} , I_8 与 I_9 合并为 I_{89} 。合并后表的行数减少。

9.3 合并效果: 冲突与错误延迟 [p128–133]

合并可能产生两个效果:

1. 可能引入 reduce-reduce conflict。
2. 不会新引入 shift-reduce conflict。
3. 错误检测可能延迟: LALR 不会比 LR 多移入错误符号, 但可能先做更多规约才发现错误。

为什么不会新引入 shift-reduce conflict? 如果合并后的状态中出现完成项 $[A \rightarrow \alpha \cdot, a]$ 和移入项 $[B \rightarrow \beta \cdot a\gamma, b]$, 由于合并要求 core 相同, 原始某个 LR(1) 状态中也会包含对应 core 的移入项, 因此 shift-reduce conflict 本来就存在, 不能说是合并新造成的。

但 reduce-reduce conflict 可能新出现。例如:

$$S \rightarrow aBc \mid bCc \mid aCd \mid bBd, \quad B \rightarrow e, \quad C \rightarrow e.$$

两个 LR(1) 状态原本可能分别在不同 lookahead 下规约 $B \rightarrow e$ 或 $C \rightarrow e$, 合并后 lookahead 集合并, 可能在同一输入符号下同时可规约为 B 和 C 。

9.4 LALR 表构造与文法关系 [p134–137]

构造 LALR 表有两种思路:

- 直接方法: 先构造完整 LR(1) 状态, 再合并同 core 状态。简单但开销大。
- 高效方法: 构造状态时就即时合并, 避免完整展开巨大的 LR(1) 自动机。

LALR(1) 表的状态数通常与 LR(0)、SLR(1) 相同, 但规约动作比 SLR(1) 更少, 因为 lookahead 更精确。

文法包含关系:

$$LR(1) \supset LALR(1) \supset SLR(1) \supset LR(0).$$

课件总结:

- 支持文法范围: $LR > LALR > SLR > LR(0)$ 。
- 状态数量: $LR > LALR = SLR = LR(0)$ 。
- 若文法是 SLR(1), 它一定也是 LALR(1) 和 LR(1)。
- 若文法是 LR(1), 不一定是 LALR(1), 因为合并可能引入 reduce-reduce conflict。
- 若文法不是 SLR(1), 仍可能是 LALR(1), 因为更精确的 lookahead 可能消除 SLR 冲突。

9.5 LL vs LR [p138–139]

LL(k) 和 LR(k) 的核心差别在于做选择的时机。

对比项	LL(k)	LR(k)
分析方向	从根到叶，预测推导	从叶到根，反向规约
选择依据	栈顶非终结符 A 与 RHS 开头 k 个终结符	栈顶已完整识别的 RHS 与其后 k 个 lookahead
选择时机	还没看到完整 RHS 就要猜产生式	等整个 RHS 已经在栈顶后再决定规约
左因子	公共前缀会造成预测困难	可以等公共前缀之后的信息出现
左递归	递归下降不适合直接处理	可以自然处理
能力	较弱	较强，课件写作 $LL < LR$

通俗地说，LL 像“开头看几眼就猜路”，LR 像“先把一整段证据收集完，再决定它是哪条产生式的右部”。所以 LR 能处理更多文法。

10 Syntax Analysis Revisit: 语法分析总复习 [p140–189]

课件后半段以 Revisit 形式复习前几讲。下面按其顺序补齐知识点。

10.1 语法分析的位置与文法定义 [p141–145]

语法分析是编译第二阶段：

- 输入：词法分析产生的 token 序列。
- 输出：分析树（parse tree）或抽象语法树（AST）。

正则表达式和有限自动机不足以表达嵌套结构，例如 $\{a^n b^n \mid n > 0\}$ ，因此需要文法，尤其是上下文无关文法 CFG。

文法四元组：

$$G = (V_T, V_N, S, \delta).$$

组成	含义
V_T	终结符集合，来自词法分析的 token，通常是 parse tree 的叶子。
V_N	非终结符集合，表示语法结构，通常是 parse tree 的内部结点。
S	开始符号。

δ 产生式集合，形式为 $LHS \rightarrow RHS$ 。

推导是从 LHS 到 RHS 应用产生式，从开始符号生成句子；规约是推导的逆过程，从输入串回到开始符号。分析树表示推导或规约的层次结构，但隐藏了具体应用产生式的先后顺序。

10.2 句型、句子、语言、二义性 [p144–145]

以文法 $A \rightarrow c \mid Ab$ 为例：

$$A \Rightarrow c, \quad A \Rightarrow Ab \Rightarrow cb, \quad A \Rightarrow Abb \Rightarrow cbb, \dots$$

可生成语言 $\{cb^n \mid n \geq 0\}$ 。

- 句型：从开始符号经过若干步推导得到的符号串，可含非终结符。
- 句子：只含终结符的句型。
- 语言：文法可生成的所有句子的集合。
- 二义文法：某个句子有不只一棵分析树，等价地有不只一个最左或最右推导。

二义性可通过定义优先级、结合性或改写文法来消除。

10.3 乔姆斯基文法体系与 AST [p146–148]

乔姆斯基将文法分为四类：

类型	名称	典型能力
Type 0	无限制文法	图灵机可识别语言
Type 1	上下文有关文法	上下文有关语言
Type 2	上下文无关文法 CFG	程序语言语法主体
Type 3	正则文法	正则语言，可由 FA 识别

AST 是 parse tree 的简化表示。它删除不影响语义的推导细节，例如只用于表达优先级和括号结构的中间非终结符，让后续语义分析和代码生成更方便。

10.4 Top-down 复习：RDP、FIRST、FOLLOW 与 LL(1) [p149–160]

Top-down parser 从根到叶构造语法树，核心问题是：

- 当前替换哪个非终结符？
- 选择该非终结符的哪条产生式？

递归下降分析（RDP）直观，但存在两个问题：

- 左递归会导致无限递归，需要消除。
- 回溯开销大且实现复杂，应尽量避免。

直接左递归消除公式：

$$A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$$

改写为：

$$A \rightarrow \beta_1 A' \mid \cdots \mid \beta_n A', \quad A' \rightarrow \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

FIRST 与 FOLLOW：

- FIRST(α)： α 能推出的句子的首个终结符集合，可包含 ε 。
- FOLLOW(A)：在某个句型中能紧跟在 A 后面的终结符集合，可包含结束符 $\$$ 。

表达式文法消左递归后：

$$\begin{aligned} E &\rightarrow TE', & E' &\rightarrow +TE' \mid \varepsilon, \\ T &\rightarrow FT', & T' &\rightarrow *FT' \mid \varepsilon, \\ F &\rightarrow (E) \mid id. \end{aligned}$$

课件给出的集合为：

$$\begin{aligned} \text{FIRST}(E) &= \{ (, id \}, & \text{FIRST}(T) &= \{ (, id \}, \\ \text{FIRST}(E') &= \{ +, \varepsilon \}, & \text{FIRST}(T') &= \{ *, \varepsilon \}, \\ \text{FIRST}(F) &= \{ (, id \}, \\ \text{FOLLOW}(E) &= \{ \$,) \}, & \text{FOLLOW}(T) &= \{ +, \$,) \}, \\ \text{FOLLOW}(E') &= \{ \$,) \}, & \text{FOLLOW}(T') &= \{ +, \$,) \}, \\ \text{FOLLOW}(F) &= \{ *, +, \$,) \}. \end{aligned}$$

LL(1) 分析表的行是非终结符，列是终结符加 $\$$ 。非递归 LL(1) 算法根据栈顶符号 X 和当前输入符号 a 决定动作：

- 若 X 是终结符且 $X = a$ ，匹配并前进。
- 若 X 是非终结符且 $M[X, a]$ 有产生式，弹出 X ，把 RHS 逆序压栈。
- 若不匹配或表项为空，报错。
- 若 $X = a = \$$ ，接受。

10.5 Bottom-up 复习 [p161–189]

Revisit 最后再次串联本讲主线：

- Shift-reduce parser 从输入规约到开始符号，是最右推导逆过程。
- Phrase、simple phrase、handle 用于说明“该规约哪里”。

- Viable prefix 说明 LR 栈内容始终没有越过句柄右端。
- LR(k) parser 用状态序列和 ACTION/GOTO 表驱动分析。
- LR(0) 用 item、CLOSURE、GOTO 构造自动机，但不看 lookahead，能力最弱。
- SLR(1) 用 FOLLOW 限制 reduce。
- LR(1) 在 item 中加入 lookahead，精确控制 reduce。
- LALR(1) 合并 LR(1) 同 core 状态，常用于实践。

11 练习题与解答 [p190-206]

题 1：正则表达式能否表达有限语言和嵌套计数语言？

判断正则表达式是否可以表达：

$$L_1 = \{x^n y z^n \mid 1 \leq n \leq 10\}, \quad L_2 = \{x^n y z^n \mid n > 0\}.$$

解答： L_1 可以。因为 n 只从 1 到 10，语言是有限集；任何有限语言都是正则语言，都可写成有限个串的并。 L_2 不可以。它要求 x 与 z 的数量任意大且相等，有限自动机没有无限计数能力，因此不是正则语言。

题 2：DFA 没有 epsilon 弧，能识别空串吗？

解答：可以。DFA 没有 ϵ 转移不代表不能接受 ϵ 。当且仅当初始状态本身是接受状态时，DFA 接受空串。

题 3：Thompson 构造 $(a \mid b)^*$ 有几个状态？

解答：8 个。按 Thompson 构造， a 与 b 各 2 个状态，选择结构增加入口/出口，Kleene star 再增加入口/出口，课件答案为 8。

题 4：无二义文法的最左推导和最右推导分析树是否一样？

解答：一样。无二义文法中，一个句子只有一棵分析树。最左推导和最右推导只是应用产生式的顺序不同，分析树会隐去推导顺序，因此得到同一棵树。

题 5：如何判断文法是 LL(1)？

对每个非终结符的候选式 $A \rightarrow \alpha \mid \beta$ ，需要满足：

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset.$$

若某个候选式可推出 ε ，例如 $\beta \Rightarrow^* \varepsilon$ ，还要满足：

$$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset.$$

通俗解释：看一个输入符号时，不能同时指向两条候选式；如果某条候选式能为空，则当输入符号属于 FOLLOW 时才选空产生式，也不能和其他候选式冲突。

题 6：LL(1) 表的行数和列数由什么决定？

解答：行数由非终结符个数决定；列数由终结符个数加 1 决定，额外的 1 是结束符号\$。

题 7：活前缀题

设句型 $aaAb$ 的最右推导为：

$$S \Rightarrow aBb \Rightarrow aaAb,$$

句柄为 aA 。该句型的活前缀是什么？

解答：活前缀不能越过句柄 aA 的右端，因此为：

$$a, \quad aa, \quad aaA.$$

题 8：找短语、直接短语与句柄

文法：

$$E \rightarrow T \mid E + T, \quad T \rightarrow F \mid T * F, \quad F \rightarrow (E) \mid a \mid b \mid c.$$

句型： $a * b + c$ 。

解答：

$$\text{短语： } a * b + c, a * b, a, b, c.$$

$$\text{直接短语： } a, b, c.$$

$$\text{句柄： } a.$$

理由是这些短语必须对应语法树中的完整子树叶子序列，而句柄是最左直接短语。

题 9：构造 LL(1) 分析表

文法：

$$S \rightarrow A, \quad A \rightarrow \varepsilon, \quad A \rightarrow bbA.$$

解答：

$$\text{FIRST}(A) = \{b, \varepsilon\}, \quad \text{FOLLOW}(S) = \{\$, \}, \quad \text{FOLLOW}(A) = \{\$, \}.$$

分析表：

	b	$\$$
S	$S \rightarrow A$	$S \rightarrow A$
A	$A \rightarrow bbA$	$A \rightarrow \varepsilon$

解释: $S \rightarrow A$ 的 FIRST 含 b 和 ε , 因此在 b 和 FOLLOW(S) 的 $\$$ 列填表; $A \rightarrow bbA$ 在 b 列, $A \rightarrow \varepsilon$ 在 FOLLOW(A) 的 $\$$ 列。

题 10: 给定文法构造 LR(1) DFA 和分析表

文法:

$$(1) S \rightarrow A, \quad (2) A \rightarrow A + A, \quad (3) A \rightarrow B ++, \quad (4) B \rightarrow y.$$

课件给出的 LR(1) 项集核心如下:

$$I_0 : S \rightarrow \cdot A, \$; A \rightarrow \cdot A + A, +/\$; A \rightarrow \cdot B ++, +/\$; B \rightarrow \cdot y, +.$$

$$I_1 : B \rightarrow y \cdot, +.$$

$$I_2 : S \rightarrow A \cdot, \$; A \rightarrow A \cdot + A, +/\$.$$

$$I_3 : A \rightarrow B \cdot ++, +/\$.$$

$$I_4 : A \rightarrow A + \cdot A, +/\$; A \rightarrow \cdot A + A, +/\$; A \rightarrow \cdot B ++, +/\$; B \rightarrow \cdot y, +.$$

$$I_5 : A \rightarrow A + A \cdot, +/\$; A \rightarrow A \cdot + A, +/\$.$$

$$I_6 : A \rightarrow B + \cdot ++, +/\$.$$

$$I_7 : A \rightarrow B ++ \cdot, +/\$.$$

对应表中关键动作:

State	y	$+$	$\$$	A	B
0	s1			2	3
1		r4			
2		s4	acc		
3		s6			
4	s1			5	3
5		r2/s4	r2		
6		s7			
7		r3	r3		

题 11: 该文法是否为 LR(1)? 如何解决?

解答: 不是 LR(1)。从表中可见状态 5 在输入 $+$ 时存在 $r2/s4$ 冲突: 既可以按 $A \rightarrow A + A$ 规约, 也可以移入 $+$ 继续形成更长的 $A + A + A$ 。

可选解决方法：

- 改写文法，明确+ 的结合性和优先级。
- 在分析器生成工具中声明冲突解决策略，例如规定+ 左结合时倾向规约，右结合时倾向移入。
- 如果语言设计允许，也可修改语法结构避免这种二义选择。

题 12：分析句子 $y++$

使用上一题文法与表，分析 $y++$ ：

状态栈	符号栈	输入	动作/跳转
0	\$	$y++\$$	$s1$
0 1	$\$y$	$++\$$	$r4 : B \rightarrow y$
0	$\$B$	$++\$$	$GOTO(0, B) = 3$
0 3	$\$B$	$++\$$	$s6$
0 3 6	$\$B+$	$+\$$	$s7$
0 3 6 7	$\$B++$	$\$$	$r3 : A \rightarrow B++$
0	$\$A$	$\$$	$GOTO(0, A) = 2$
0 2	$\$A$	$\$$	acc

因此 $y++$ 是该文法的一个句子。

题 13：LR(0)、SLR(1)、LR(1)、LALR(1) 的关系

解答：

$$LR(1) \supset LALR(1) \supset SLR(1) \supset LR(0).$$

能力越往左越强。状态数一般为：

$$LR(1) > LALR(1) = SLR(1) = LR(0).$$

LR(0) 不看展望符号；SLR(1) 用 FOLLOW 限制规约；LR(1) 用 item 自带 lookahead 精确规约；LALR(1) 合并 LR(1) 同 core 状态，在能力和表规模之间折中。

复习建议

本讲复习时建议按三层掌握：

1. 概念层：能解释 handle、viable prefix、item、closure、goto、lookahead、core。
2. 机制层：能手工走一遍 shift-reduce 分析，能根据 ACTION/GOTO 表分析输入串。

3. 构造层：能从小文法构造 LR(0) 项集和表，能说明 SLR(1)、LR(1)、LALR(1) 分别如何改进 LR(0)。

最容易混淆的点是：FOLLOW 是 SLR(1) 的粗粒度全局信息；LR(1) item 的 lookahead 是带上下文的局部信息；LALR(1) 则把 LR(1) 的局部信息合并到同 core 状态中，所以表小但可能产生 reduce-reduce conflict。